

how following the discipline of FP is tremendously beneficial because of the increase in *modularity* that we gain from programming with pure functions. Because of their modularity, pure functions are easier to test, reuse, parallelize, generalize, and reason about. Furthermore, pure functions are much less prone to bugs.

In this chapter, we'll look at a simple program with side effects and demonstrate some of the benefits of FP by removing these side effects. We'll also discuss the benefits of FP more generally and define two important concepts—*referential transparency* and the *substitution model*.



1.1 The benefits of FP: a simple example

Let's look at an example that demonstrates some of the benefits of programming with pure functions. The point here is just to illustrate some basic ideas that we'll return to throughout this book. This will also be your first exposure to Scala's syntax. We'll talk through Scala's syntax much more in the next chapter, so don't worry too much about following every detail. As long as you have a basic idea of what the code is doing, that's what's important.

1.1.1 A program with side effects

Suppose we're implementing a program to handle purchases at a coffee shop. We'll begin with a Scala program that uses side effects in its implementation (also called an *impure* program).

Listing 1.1 A Scala program with side effects

```
class Cafe {
  def buyCoffee(cc: CreditCard): Coffee = {
    val cup = new Coffee()
    cc.charge(cup.price)
    cup
  }
}
```

The `class` keyword introduces a class, much like in Java. Its body is contained in curly braces, { and }.

A method of a class is introduced by the `def` keyword.

`cc: CreditCard` defines a parameter named `cc` of type `CreditCard`. The `Coffee` return type of the `buyCoffee` method is given after the parameter list, and the method body consists of a block within curly braces after an `=` sign.

Side effect. Actually charges the credit card.

No semicolons are necessary. Newlines delimit statements in a block.

We don't need to say `return`. Since `cup` is the last statement in the block, it is automatically returned.

The line `cc.charge(cup.price)` is an example of a side effect. Charging a credit card involves some interaction with the outside world—suppose it requires contacting the credit card company via some web service, authorizing the transaction, charging the

card, and (if successful) persisting some record of the transaction for later reference. But our function merely returns a `Coffee` and these other actions are happening *on the side*, hence the term “side effect.” (Again, we’ll define side effects more formally later in this chapter.)

As a result of this side effect, the code is difficult to test. We don’t want our tests to actually contact the credit card company and charge the card! This lack of testability is suggesting a design change: arguably, `CreditCard` shouldn’t have any knowledge baked into it about how to contact the credit card company to actually execute a charge, nor should it have knowledge of how to persist a record of this charge in our internal systems. We can make the code more modular and testable by letting `CreditCard` be ignorant of these concerns and passing a `Payments` object into `buyCoffee`.

Listing 1.2 Adding a payments object

```
class Cafe {
  def buyCoffee(cc: CreditCard, p: Payments): Coffee = {
    val cup = new Coffee()
    p.charge(cc, cup.price)
    cup
  }
}
```

Though side effects still occur when we call `p.charge(cc, cup.price)`, we have at least regained some testability. `Payments` can be an interface, and we can write a mock implementation of this interface that is suitable for testing. But that isn’t ideal either. We’re forced to make `Payments` an interface, when a concrete class may have been fine otherwise, and any mock implementation will be awkward to use. For example, it might contain some internal state that we’ll have to inspect after the call to `buyCoffee`, and our test will have to make sure this state has been appropriately modified (*mutated*) by the call to `charge`. We can use a *mock framework* or similar to handle this detail for us, but this all feels like overkill if we just want to test that `buyCoffee` creates a charge equal to the price of a cup of coffee.

Separate from the concern of testing, there’s another problem: it’s difficult to reuse `buyCoffee`. Suppose a customer, Alice, would like to order 12 cups of coffee. Ideally we could just reuse `buyCoffee` for this, perhaps calling it 12 times in a loop. But as it is currently implemented, that will involve contacting the payment system 12 times, authorizing 12 separate charges to Alice’s credit card! That adds more processing fees and isn’t good for Alice or the coffee shop.

What can we do about this? As the figure at the top of page 6 illustrates, we could write a whole new function, `buyCoffees`, with special logic for batching up the charges.¹ Here, that might not be such a big deal, since the logic of `buyCoffee` is so

¹ We could also write a specialized `BatchingPayments` implementation of the `Payments` interface, that somehow attempts to batch successive charges to the same credit card. This gets complicated though. How many charges should it try to batch up, and how long should it wait? Do we force `buyCoffee` to indicate that the batch is finished, perhaps by calling `closeBatch`? And how would it know when it’s appropriate to do that, anyway?